



UNIVERSIDADE FEDERAL DO CEARÁ

RELATÓRIO: TRABALHO 3 E 4

Aluno:

Francisco Werley da Silva - 553948

Camile Isidorio Araújo - 555251

Professor: Rafael Braga

Disciplina: Sistemas Distribuídos

1. Introdução

- O que aprenderam e o que ainda tem dificuldade?
 - Aprendemos melhor como funciona a comunicação indireta;
 - Sem muitas dificuldades.
- Nota que a dupla merece (em uma escala de 0 a 10);
 - 10
- Nível de dificuldade (muito fácil, fácil, média, difícil ou muito difícil);
 - média/difícil
- Quantas horas precisaram para completar a atividade? (Total = estudo pessoal + leituras + videoaulas extra + implementação)
 - 6 horas em média
- Caso desejem, deixem alguma sugestão para melhorar a atividade.
 - sem sugestões
- Qual o link do repositório com o código-fonte?
 - [LINK](#)

2. Descrição da implementação

Este trabalho apresenta o desenvolvimento de uma aplicação de Gestão de Recursos Humanos em Java, com foco em sistemas distribuídos e comunicação entre processos em linguagens distintas. Diferentemente do trabalho anterior, em que a comunicação era feita por Java RMI com protocolo request-reply e um único cliente Java, aqui a arquitetura foi ampliada para contemplar dois paradigmas de comunicação distribuída: comunicação direta, por meio de API REST sobre HTTP, e comunicação indireta, por meio de filas de mensagens persistentes com RabbitMQ, conforme a Opção C do enunciado (Message Queues). O servidor permanece centralizado em Java com Spring Boot, enquanto os clientes foram implementados em JavaScript (Node.js) e Python, demonstrando interoperabilidade entre plataformas heterogêneas.

A modelagem do domínio partiu da superclasse abstrata Colaborador, com atributos id, nome e salário, e das subclasses Funcionario, Estagiario, Autonomo e Efetivo, permitindo especialização e reaproveitamento por herança. A classe Efetivo estende Funcionario e adiciona bônus anual e anos de empresa, formando uma hierarquia de três níveis. A interface Admissivel, implementada por Funcionario e Efetivo, define o comportamento de admissão e permite polimorfismo na exibição de mensagens de admissão. A classe Departamento representa agregação: possui um gerente (Colaborador) e uma lista de colaboradores membros, além de calcular a folha somando o custo total de cada membro.

Cada tipo de colaborador implementa `calcularCustoTotal()` de forma distinta. `Funcionario` e `Efetivo` aplicam encargos de 68% sobre o salário; `Estagiario` soma vale-transporte fixo de R\$200,00 e seguro equivalente a 5% do salário; `Autonomo` aplica acréscimo de 5%. No caso de `Efetivo`, o bônus anual é rateado em doze parcelas e somado ao custo mensal. Esse polimorfismo é utilizado tanto localmente quanto nas operações remotas de cálculo de folha expostas pela API.

A arquitetura do servidor foi dividida em seis pacotes sob o namespace `com.rh.rh`. O pacote `model` concentra as entidades de domínio e as anotações Jackson (`@JsonTypeInfo` e `@JsonSubTypes`) que permitem serialização e desserialização polimórfica em JSON com base no campo `tipo`. O pacote `controller` abriga `ColaboradorController`, `DepartamentoController` e `FolhaController`, responsáveis por mapear os endpoints REST. O pacote `service` contém `ColaboradorService`, `DepartamentoService` e `FolhaService`, onde reside a lógica de negócio. O pacote `repository` implementa `ColaboradorRepository` e `DepartamentoRepository` com armazenamento em `ConcurrentHashMap`. O pacote `messaging` reúne `RabbitMQConfig`, `RhEvent` e `EventoService`, dedicado à comunicação indireta. O pacote `exception` contém `ResourceNotFoundException` e `GlobalExceptionHandler` para tratamento centralizado de erros HTTP.

A comunicação direta foi implementada como API REST com Spring Web MVC. O servidor escuta na porta 8080 e expõe recursos organizados por caminhos: `/colaboradores` para CRUD e cálculo de custo individual; `/departamentos` para CRUD e associação de colaboradores; e `/folha` para consultas agregadas de folha de pagamento. O protocolo segue o modelo requisição-resposta do HTTP: o cliente envia um verbo (GET, POST ou DELETE), o servidor processa e retorna status e corpo em JSON. Ao adicionar um colaborador, por exemplo, o cliente serializa o objeto com o campo `tipo` indicando a subclasse, envia via POST, e o Jackson reconstrói a instância correta (`Funcionario`, `Estagiario`, `Autonomo` ou `Efetivo`) antes de armazená-la no repositório. Isso caracteriza passagem por valor: o cliente não possui referência ao objeto no servidor, apenas uma cópia serializada trafega pela rede. Respostas de erro seguem formato padronizado com campo `erro` (HTTP 400 para argumentos inválidos, HTTP 404 para recursos inexistentes), tratadas pelo `GlobalExceptionHandler`.

Foram expostos quinze endpoints REST distribuídos em três controllers. `ColaboradorController` oferece listar, buscar por id, adicionar, remover e consultar custo mensal. `DepartamentoController` oferece listar, buscar por id, criar, remover e adicionar colaborador a um departamento. `FolhaController` oferece folha total da empresa, folha por departamento, custo por colaborador e resumo de custos agrupados por tipo. `DepartamentoService` depende de `ColaboradorService` para resolver colaboradores por id ao montar departamentos, mantendo a lógica de negócio separada da modelagem de entidades e do acesso a dados.

A comunicação indireta foi implementada com RabbitMQ, atendendo ao requisito mínimo da Opção C: resiliência a falhas, de modo que, se o componente receptor estiver offline, a mensagem aguarda na fila até que ele retorne. O broker é provisionado via docker-compose na porta 5672. O servidor declara um exchange fanout durável chamado `rh.eventos`. Após cada operação de escrita nos services (criação ou remoção de colaborador, criação ou remoção de departamento, associação de colaborador a departamento), o `EventoService` publica um `RhEvent` contendo tipo, origem, timestamp e dados da operação. O

exchange fanout replica cada evento para todas as filas vinculadas, sem que o remetente precise conhecer os destinatários, caracterizando desacoplamento espacial.

Cada cliente mantém sua própria fila durável: `rh.eventos.javascript` para o cliente Node.js e `rh.eventos.python` para o cliente Python. Essa separação garante que ambos recebam todos os eventos e que mensagens destinadas a um cliente permaneçam armazenadas caso ele esteja desconectado no momento da publicação. Os eventos publicados incluem `COLABORADOR_CRIADO`, `COLABORADOR_REMOVIDO`, `DEPARTAMENTO_CRIADO`, `DEPARTAMENTO_REMOVIDO` e `COLABORADOR_ADICIONADO_DEPARTAMENTO`. Operações de leitura via GET não geram eventos, pois não alteram o estado do sistema.

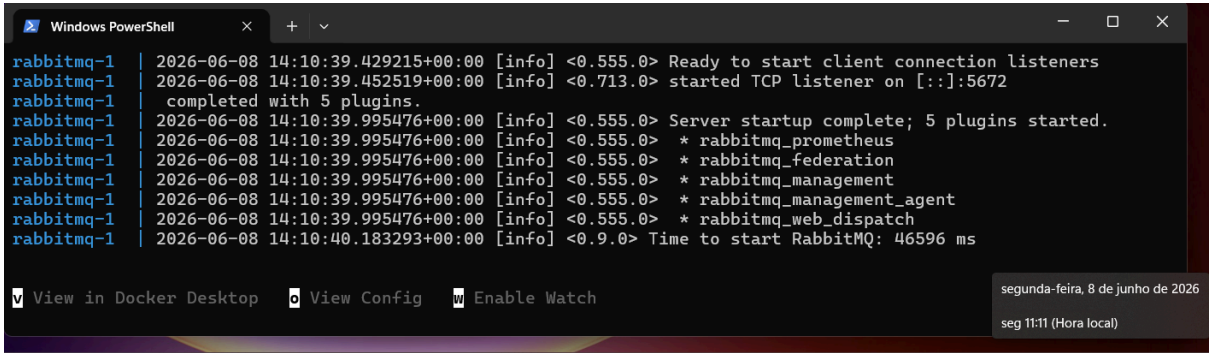
O cliente JavaScript (`client.js`) consome a API REST por meio de fetch nativo do Node.js, com menu interativo de doze opções em modo texto. Paralelamente, o módulo `queue.js` estabelece conexão com o RabbitMQ via biblioteca `amqp-lib`, declara a fila durável, vincula-a ao exchange e consome mensagens em background, exibindo eventos com o prefixo [FILAS]. O cliente Python (`client.py`) segue a mesma estrutura: requisições HTTP com a biblioteca `requests` para operações síncronas, e o módulo `queue_listener.py` executa o consumidor em thread separada com a biblioteca `pika`. Os dois clientes não compartilham código, não se comunicam diretamente entre si e podem operar simultaneamente sobre o mesmo servidor, compartilhando o estado mantido em memória pelo repositório Java.

O servidor inicia com quatro colaboradores pré-cadastrados no `ColaboradorRepository`: um `Funcionario`, um `Estagiario`, um `Autonomo` e um `Efetivo`, permitindo testes imediatos de listagem, cálculo de folha e polimorfismo JSON. Para executar, inicia-se o RabbitMQ com `docker compose up -d`, compila-se e executa-se o servidor com Maven (`mvnw spring-boot:run`), e, em terminais separados, executam-se `node client.js` e `python client.py` após instalar as dependências (`npm install` e `pip install -r requirements.txt`).

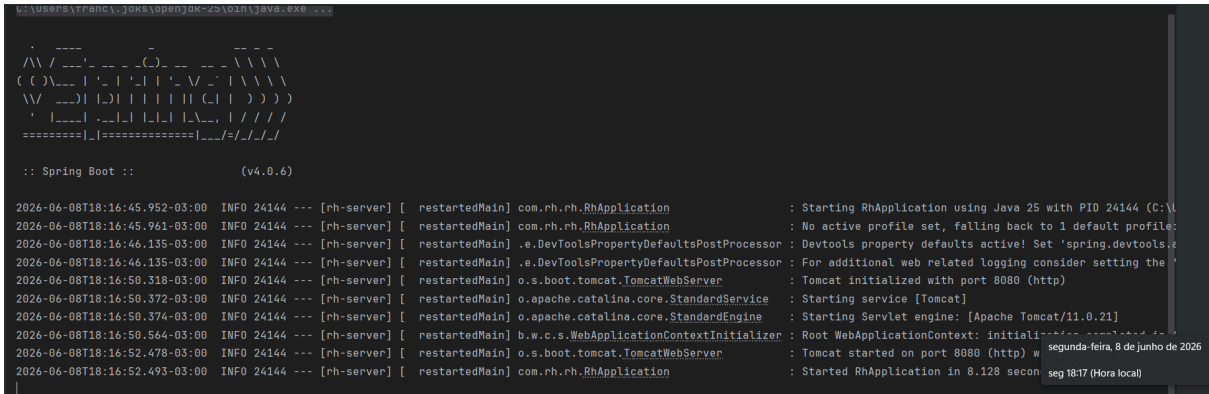
Como resultado, o trabalho consolidou a implementação prática de dois modelos de comunicação distribuída no mesmo sistema. A API REST representa comunicação direta, síncrona e acoplada ao endereço do servidor, adequada para operações CRUD com resposta imediata. As filas RabbitMQ representam comunicação indireta, assíncrona e desacoplada, na qual produtores e consumidores não precisam estar ativos simultaneamente nem conhecer uns aos outros. A separação em camadas (model, controller, service, repository, messaging) facilitou a organização e a manutenção. Em relação ao trabalho anterior com RMI, a abordagem REST elimina a dependência de Registry e stubs Java, permitindo clientes em linguagens arbitrárias, enquanto o RabbitMQ introduz um intermediário que garante persistência de mensagens e notificação assíncrona entre clientes heterogêneos. A abordagem atende aos requisitos do enunciado: comunicação distribuída com múltiplos clientes em linguagens distintas, modelagem OO com herança e agregação, interface em modo texto, comunicação direta via API REST e comunicação indireta via filas de mensagens com resiliência a falhas do receptor.

Prints:

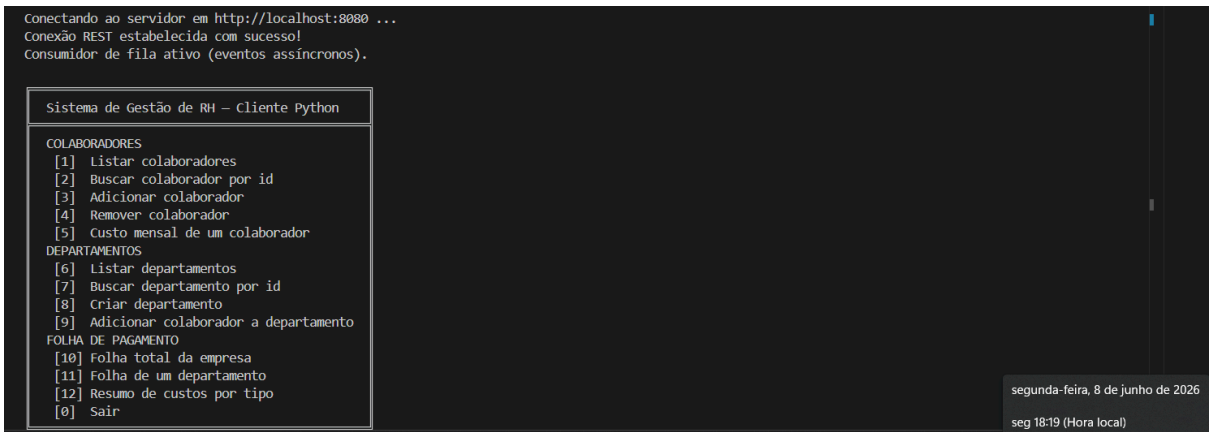
RabbitMQ:



Servidor Java:



Cliente Python:



Cliente JavaScript:

Conectando ao servidor em http://localhost:8080 ...
> Conexão REST estabelecida com sucesso!
Consumidor de fila ativo (eventos assíncronos).

Sistema de Gestão de RH – Cliente JavaScript

COLABORADORES

[1] Listar colaboradores
[2] Buscar colaborador por id
[3] Adicionar colaborador
[4] Remover colaborador
[5] Custo mensal de um colaborador

DEPARTAMENTOS

[6] Listar departamentos
[7] Buscar departamento por id
[8] Criar departamento
[9] Adicionar colaborador a departamento

FOLHA DE PAGAMENTO

[10] Folha total da empresa
[11] Folha de um departamento
[12] Resumo de custos por tipo
[0] Sair

segunda-feira, 8 de junho de 2026

seg 18:20 (Hora local)